

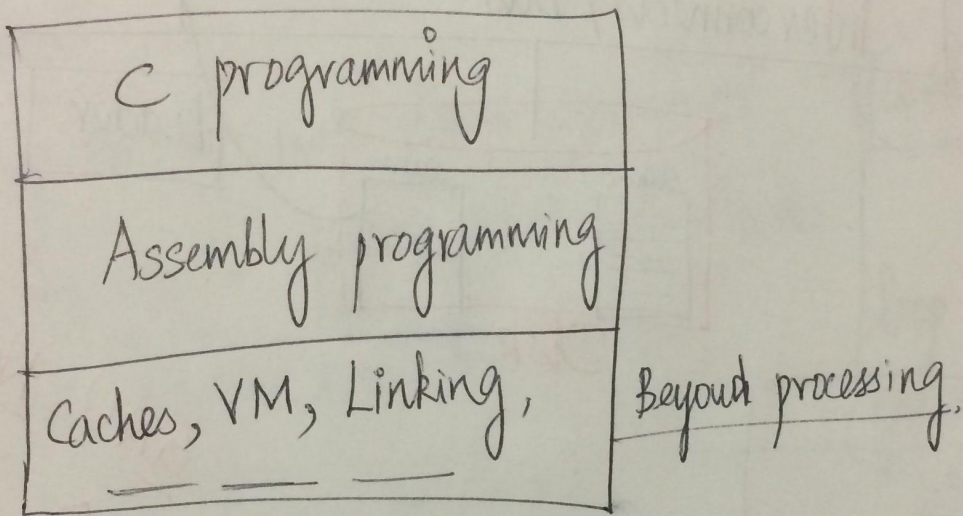
CS 354

①

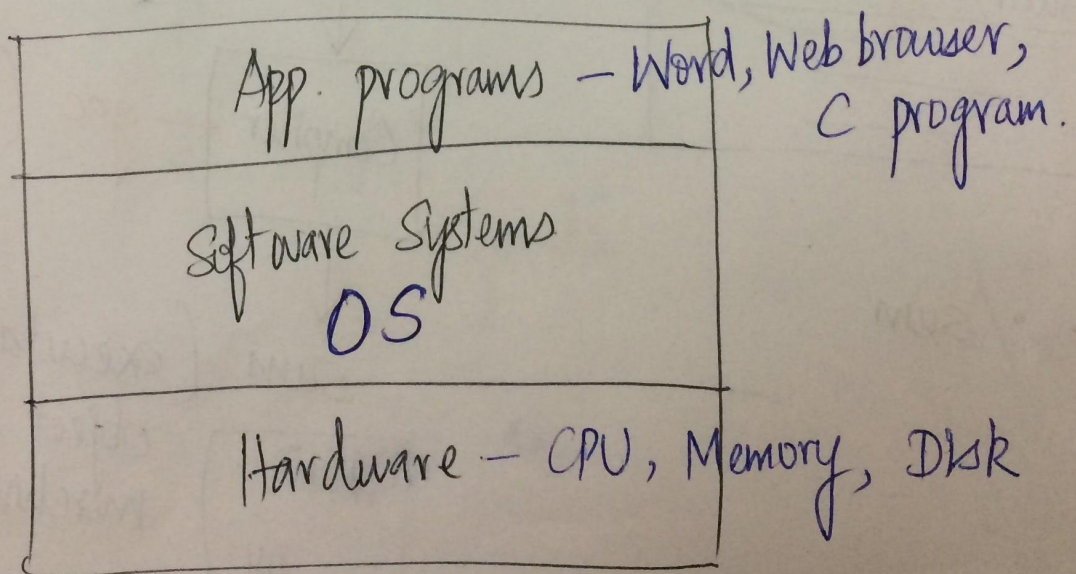
1. How computers work?

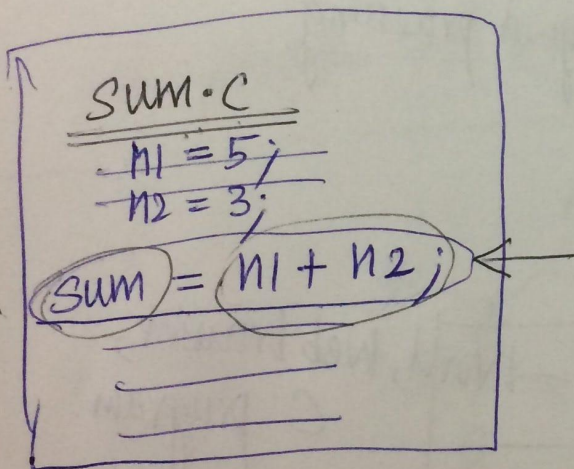
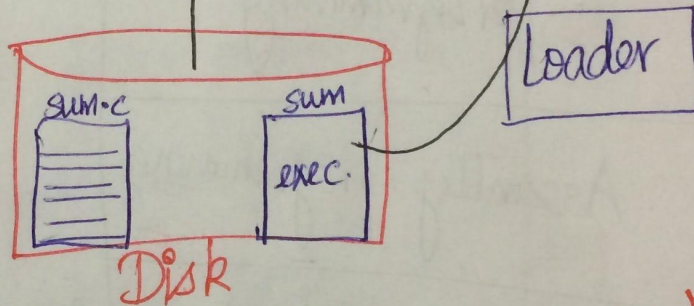
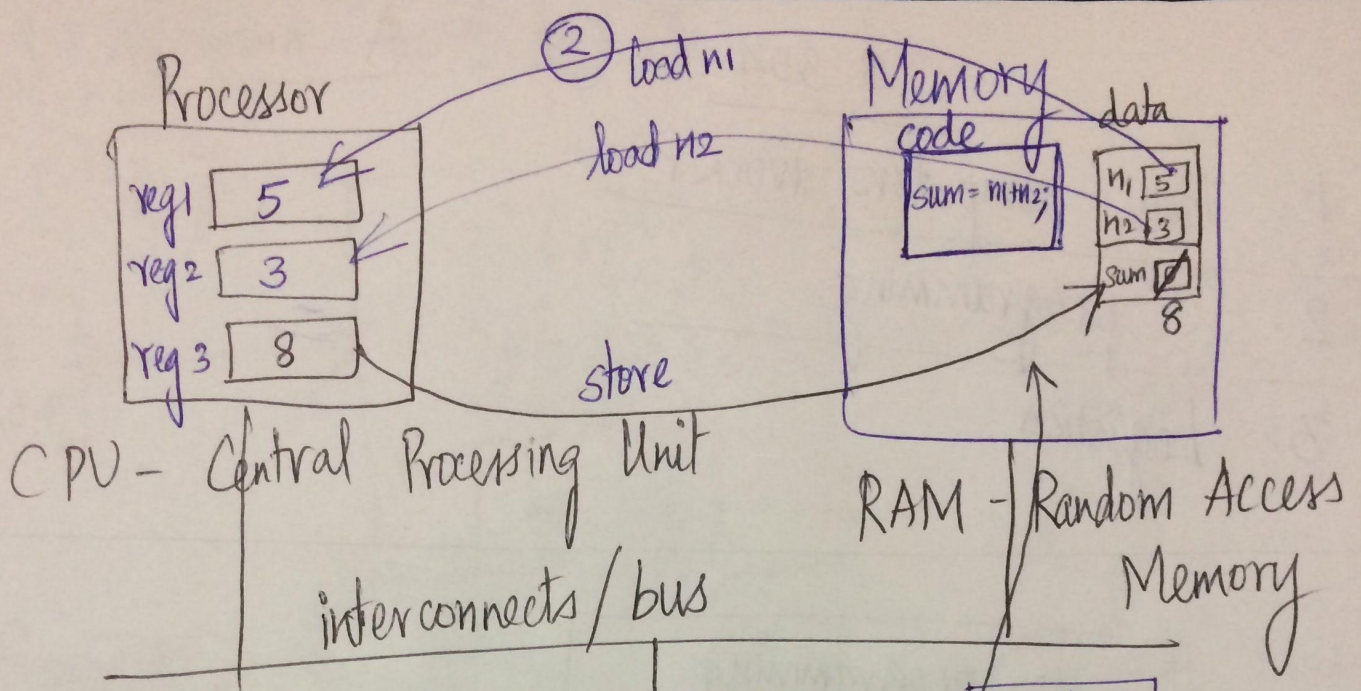
2. C programming

3. Logistics

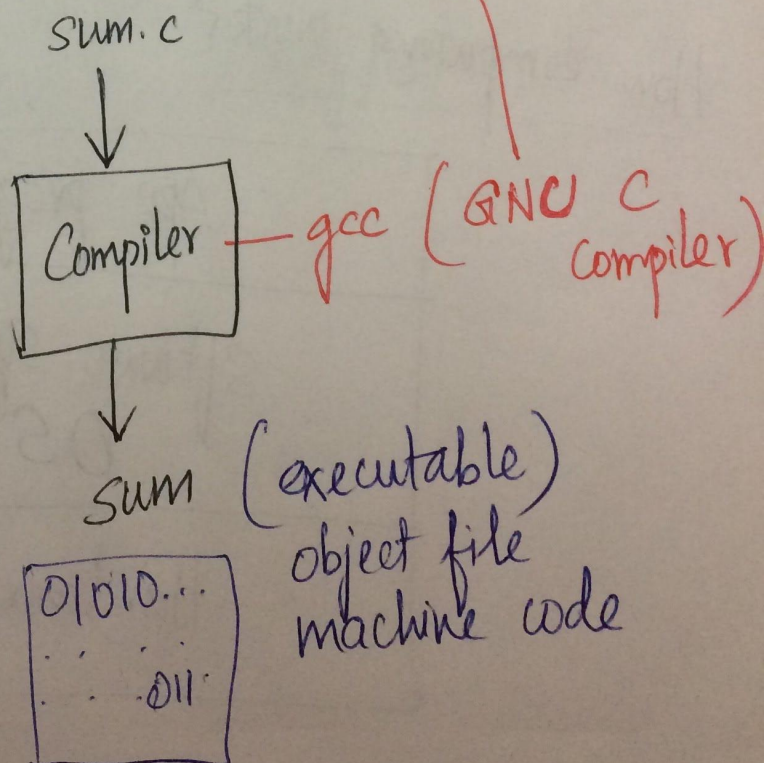


How computers work?



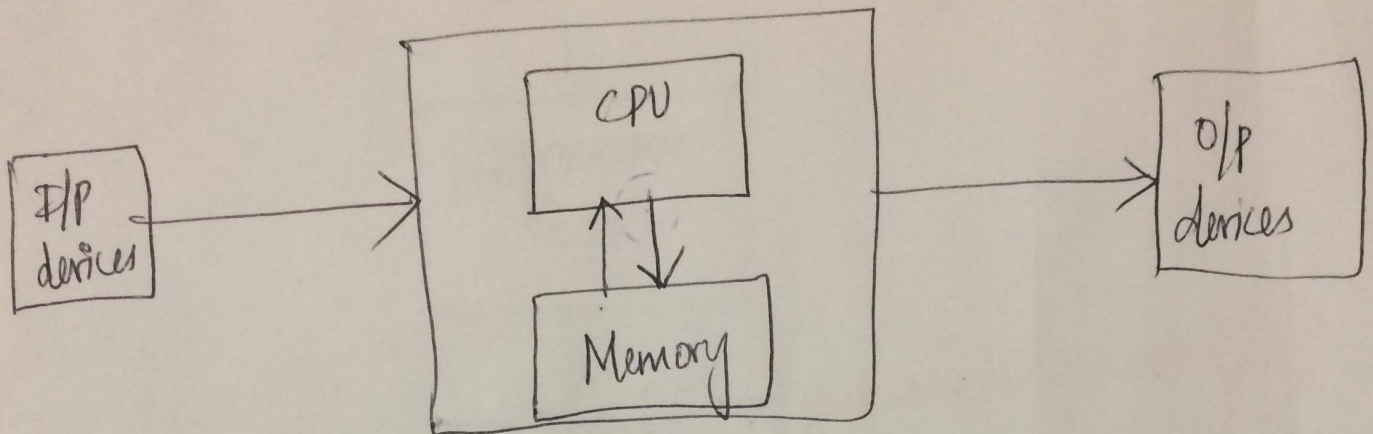


> ./sum



GNU Not Unix

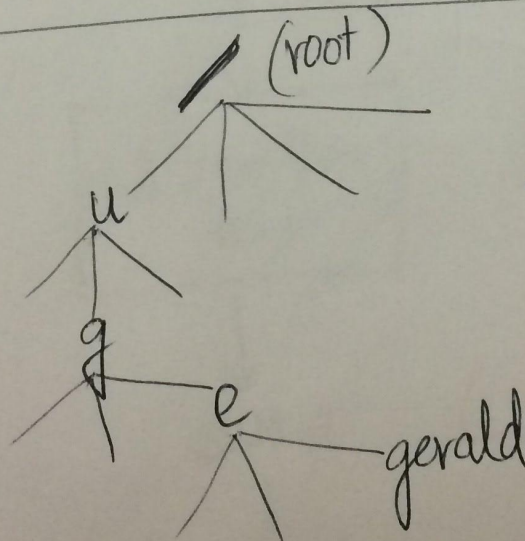
Von Neumann Architecture ③



Processor

1. fetch - instr from memory
2. decode - figure out what to do.
3. execute - run this instr.

add reg1, reg2
loop
(get next instr.)

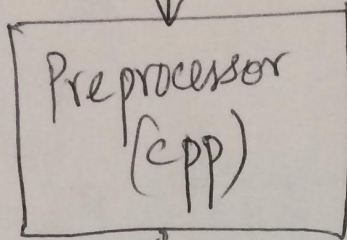


/u/g/e/gerald

Build Process

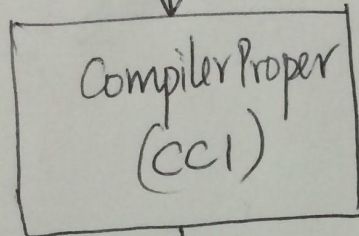
sum.c^④

source file



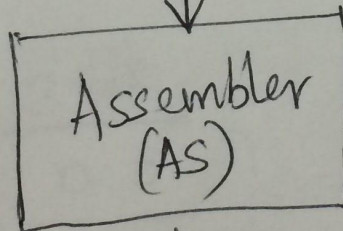
sum.i

(preprocessed source file)



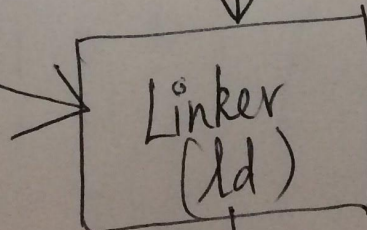
sum.s

(Assembly file)



sum.o

relocatable
(object file)



sum

executable file.

printf.o

→

C Programming ⁽⁵⁾

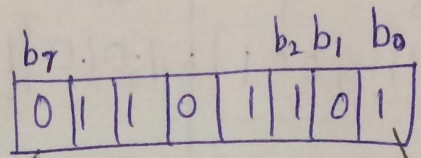
bits - 0 OR 1

(binary)

circuit
high voltage
low voltage

0

1 byte = 8 bits



2 bits $\Rightarrow$ 4 values
 $= 2^2$

0	0	$\rightarrow$ 0
0	1	$\rightarrow$ 1
1	0	$\rightarrow$ 2
1	1	$\rightarrow$ 3

$\begin{matrix} 2^1 & 2^0 \end{matrix}$

$$21 = 2 \times \underline{10}^1 + 1 \times \underline{10}^0 = 20 + 1 = 21$$

$$1 \times \underline{2}^1 + 1 \times \underline{2}^0 = 2 + 1 = 3$$

3 bits

0	0	0
0	0	1
0	1	0
0	1	1
1	0	0
1	0	1
1	1	0
1	1	1

8 = 2^3 value

4 bits

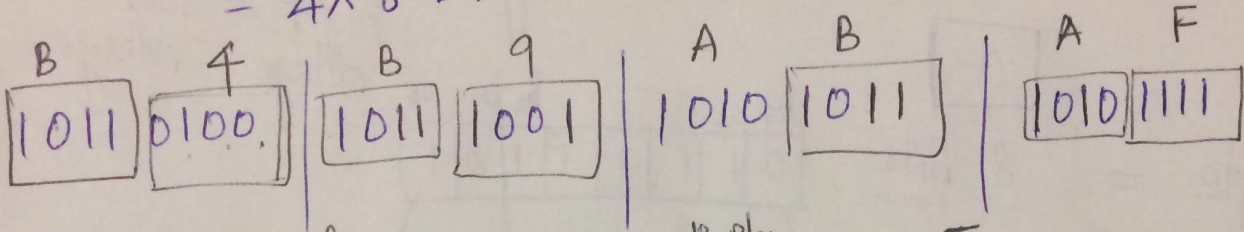
$2^4 = 16$ values

int sum;

⑥

$$[4 \text{ bytes}] = \underline{\underline{32 \text{ bits}}}$$

$$= 4 \times 8 \text{ bits}$$



Hexadecimal $\rightarrow$ 16 digits

6

10

0-9, A, B, C, D, E, F

4 bits

$$\begin{array}{c} \overset{3}{2}(8) \quad \overset{2}{2}(4) \quad \overset{1}{2}(2) \quad \overset{0}{2}(1) \\ \hline 0 \quad 0 \quad 0 \quad 0 \\ \hline 0 \quad 0 \quad 0 \quad 1 \end{array}$$

$$1 \quad 0 \quad 0 \quad 1 \Rightarrow 9$$

$$1 \quad 0 \quad 1 \quad 0 \Rightarrow A$$

$$1 \quad 1 \quad 1 \quad 1 \Rightarrow F$$

int in hexadee. = 0x B4 B9 AB AF

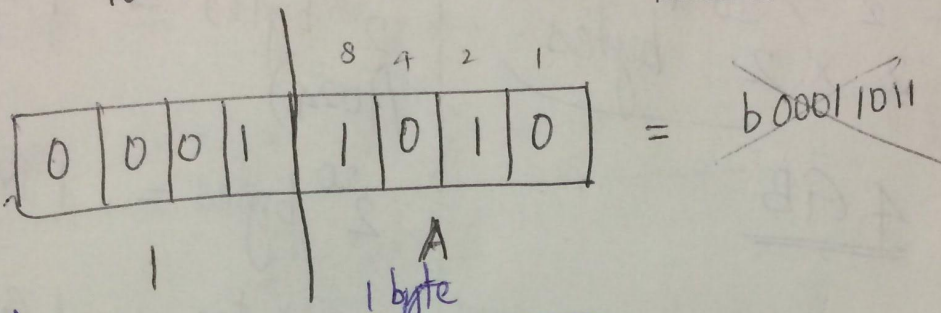
Zero small x

Memory

(7)

hex $\longrightarrow$ decimal

$$\textcircled{0x1A} = 1 \times 16^1 + 10 \times 16^0$$
$$\downarrow \quad \downarrow$$
$$16^1 \quad 16^0 = 16 + 10 = \boxed{26}$$



addr

} 8 bits

if 4 bits,
memory locations
0-15

16 bytes

starting
addr $\longrightarrow$

0	
1	
2	
3	
4	0000 0000
5	0000 0000
6	0000 0000
7	0000 1011
...	
15	

num
= 0x0000000B

num = 11

1011

(8)

32-bit machine

32-bits $\rightarrow$ addr

2^{32} memory locations/ addrs.
 $= 2^2 \times 2^{30}$ bytes

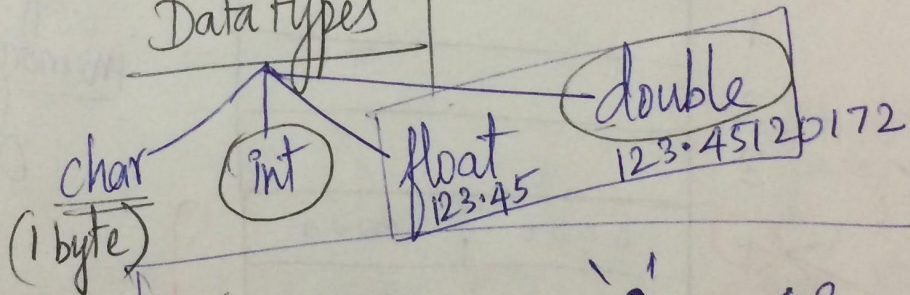
4 GB

2^{10} bytes = 1 KB
(1024)

2^{20} bytes = 1 MB

2^{30} bytes = 1 GB

Data types



char
char c = 'A';

c = c + 1;
 $\downarrow$
'B'

'A' - 65

'0' - 48

'a' - 97

'b' - 98

ASCII encoding

$$0x41 = 16 \times 4 + 1 \times 1 = 64 + 1 = \underline{\underline{65}}$$

$\downarrow \quad \downarrow$
 $16^1 \quad 16^0$

strings

(9)

char array + something more

`'E' 'P' 'I' 'C' '\0'`

`char[5] = "EPIC";`

`char ch = 'A';` → NUL terminator. → `'A'`

`char ch[2] = "A";` → `'A' '\0'`

Operators

Arithmetic

`+, -, *, /, %`

Relational

`<, <=, >, >=`
`==, !=`

Logical

`&&, ||, !`

bitwise

`&, |, ^`
AND OR EX-OR

incr/decr
`++, --`

assignment
`=, +=, -=`

`3 * 4 / 5`

`3 * (4 / 5)`

Decisions

if (expr)
 stmt1;

if (expr) {

if (expr) {

} else {

}

}
if (expr) {

"switch"

} else if (expr1) {

} else {

No boolean in C.

0 $\Rightarrow$ F

non-zero $\Rightarrow$ T

}
if (X = 10)

intention

X == 10

if (expr)
 stmt1;
 stmt2;

~~else~~ ~~stmt3;~~

Short circuiting

(11)

X	Y	$X \&\& Y$	$X Y$
T	T	T	T
T	F	F	T
F	T	F	T
F	F	F	F

if (~~X~~ X && Y)

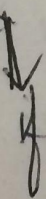


if X is F

$\Rightarrow X \&\& Y$ is F

$\therefore$ don't evaluate Y

if (X || Y)



if X is T $\Rightarrow$ don't evaluate Y.

eg. d = 0;

if (d != 0 && num/d) {

}

Loops

int sum (int x, int y); (13)

main() {

~~int s = x + y;~~ sum (7, 5);

}
int sum (int x, int y) {
int z = x + y;

return (z); → don't use this name!

}

main() {

x = 0;

fn(x);

→ print x = ?

}

void fn (int x) {

x = 1;

}